Modified SAP-1 CPU in Verilog HDL

CPE223 Computer Architecture Mini-Project

Project Members

Name	Student ID
Kiatisak Markmeeshap	67070501005
Tithinan Sobking	67070501018
Thanaboon Tikaew	67070501021
Worawut Sereethai	67070501040
Chanon Lhumsa-ard	67070501059
Siriwan Yindeephot	67070501075

Abstract

This mini-project presents a modified SAP-1 CPU designed in Verilog HDL. The processor keeps the simple structure of the original SAP-1 model while extending it with additional arithmetic instructions. It uses an 8-bit datapath, 4-bit addresses, 16 memory locations, and an 8-bit instruction format made from a 4-bit opcode and a 4-bit operand. The supported instructions are LDA, ADD, SUB, MUL, DIV, OUT, and HLT. Multiplication and division are implemented manually without using the Verilog `*` and `/` operators. Functional simulation confirms that the CPU executes the main program correctly and handles arithmetic edge cases, division by zero, and halt behavior as expected.

1 System Architecture and Datapath

The CPU follows a multi-cycle SAP-1 style architecture. Instead of completing an instruction in one clock cycle, the processor breaks each instruction into smaller fetch, decode, and execute steps. This makes the control flow easier to observe and matches the educational purpose of the SAP-1 computer. The datapath uses a multiplexer-based 8-bit shared bus rather than an internal tri-state bus, which makes the implementation more suitable for FPGA synthesis.

Component	Verilog File
Program Counter	`program_counter.v`
MAR	`memory_address_register.v`
RAM	`ram_16x8.v`
Instruction Register	`instruction_register.v`
Accumulator	`accumulator.v`
B Register	`b_register.v`
ALU	`alu.v`
Output Register	`output_register.v`
Control Unit	`control_unit.v`
Bus Multiplexer	`bus_mux.v`
Top-level CPU	`sap1_top.v`
Testbench	`sap1_tb.v`

The top-level module connects the datapath, control unit, RAM programming interface, and external outputs:

```
module sap1_top(
    input clk,
    input reset,
    input prog_we,
    input [3:0] prog_addr,
    input [7:0] prog_data,
    output [7:0] out,
    output halted,
    output reg div_zero
);
```

Listing 1: Top-level interface from sap1_top.v

2 Module Descriptions

This section explains each Verilog module and the role it plays in the SAP-1 datapath. The modules are intentionally kept small so that the flow of data through the CPU can be followed clearly.

2.1 Program Counter

The Program Counter in `program_counter.v` is a 4-bit register that points to the next instruction in memory. During `FETCH_ADDR`, its value is placed on the bus and loaded into the MAR. During `FETCH_INST`, the control unit asserts `pc_inc`, allowing the PC to move to the next instruction address. The module also includes reset and load support, although normal execution mainly uses reset and increment.

```
if (reset)
    pc <= 4'h0;
else if (load)
    pc <= data_in;
else if (inc)
    pc <= pc + 4'h1;
```

Listing 2: Program Counter update logic

2.2 Memory Address Register

The Memory Address Register in `memory_address_register.v` stores the 4-bit address currently sent to RAM. During instruction fetch, it receives the PC value. During memory-based instructions such as LDA, ADD, SUB, MUL, and DIV, it receives the operand field from the instruction register. In this way, the MAR acts as the address bridge between the control sequence and the RAM.

```
if (reset)
    address <= 4'h0;
else if (load)
    address <= data_in;
```

Listing 3: MAR load logic

2.3 RAM

The RAM in `ram_16x8.v` contains 16 memory locations, each 8 bits wide. It stores both program instructions and data values, following the simple memory model used by SAP-1. The CPU reads RAM through the current MAR address. Before execution begins, the testbench loads instructions and data through the programming interface `prog_we`, `prog_addr`, and `prog_data`.

```
reg [7:0] memory [0:15];
assign data_out = memory[address];

always @(posedge clk) begin
    if (prog_we)
        memory[prog_addr] <= prog_data;
end
```

Listing 4: RAM storage and program loading

2.4 Instruction Register

The Instruction Register in `instruction_register.v` stores the 8-bit instruction fetched from RAM. It separates the instruction into two fields: `instruction[7:4]` as the opcode and `instruction[3:0]` as the operand. The opcode is sent to the control unit, while the operand is used as a RAM address for memory-reference instructions.

```
if (reset)
    instruction <= 8'h00;
else if (load)
    instruction <= data_in;

assign opcode = instruction[7:4];
assign operand = instruction[3:0];
```

Listing 5: Instruction Register field extraction

2.5 Accumulator

The Accumulator in `accumulator.v` is the main 8-bit working register of the CPU. LDA loads RAM data directly into the accumulator. Arithmetic instructions use the accumulator as the first ALU input, and the ALU result is written back into the accumulator. Thus, most data processing operations update this register.

```
if (reset)
    data_out <= 8'h00;
else if (load)
    data_out <= data_in;
```

Listing 6: Accumulator register logic

2.6 B Register

The B Register in `b_register.v` stores the second ALU operand. For ADD, SUB, MUL, and DIV, the CPU first loads `RAM[operand]` into the B register during `ALU_READ`. In the next state, the ALU operates on the accumulator and B register values.

```
if (reset)
    data_out <= 8'h00;
else if (load)
    data_out <= data_in;
```

Listing 7: B Register load logic

2.7 ALU

The ALU in `alu.v` performs ADD, SUB, MUL, and DIV operations based on the opcode. ADD and SUB use standard 8-bit arithmetic. MUL is implemented using shift-and-add logic, and DIV is implemented using restoring division based on shift/subtract operations. Division by zero is explicitly handled by returning `8'hFF` and asserting `div_zero`.

```
case (opcode)
    OP_ADD: result = a + b;
    OP_SUB: result = a - b;
    OP_MUL: result = multiply_shift_add(a, b);
    OP_DIV: begin
        if (b == 8'h00) begin
            result = 8'hFF;
            div_zero = 1'b1;
        end else begin
            result = divide_restoring(a, b);
        end
    end
    default: result = a;
endcase
```

Listing 8: ALU operation selection

2.8 Output Register

The Output Register in `output_register.v` stores the value produced by the OUT instruction. When OUT executes, the control unit selects the accumulator onto the bus and asserts `out_load`. The output register then holds this value as the external CPU output.

```
if (reset)
    data_out <= 8'h00;
else if (load)
    data_out <= data_in;
```

Listing 9: Output Register load logic

2.9 Control Unit

The Control Unit in `control_unit.v` is an FSM that coordinates all CPU operations. It generates the bus select signal, register load enables, PC increment signal, ALU opcode, and halted output. Its states implement instruction fetch, opcode decode, memory address setup, data read, ALU write-back, output write, and halt.

```
always @(posedge clk or posedge reset) begin
    if (reset)
        state <= FETCH_ADDR;
    else
        state <= next_state;
end

bus_sel = BUS_ZERO;
mar_load = 1'b0;
ir_load = 1'b0;
pc_inc = 1'b0;
a_load = 1'b0;
b_load = 1'b0;
out_load = 1'b0;
```

Listing 10: Control Unit state register and default controls

2.10 Bus Multiplexer

The Bus Multiplexer in `bus_mux.v` implements the shared 8-bit datapath. It selects one source at a time from PC, RAM data, IR operand, accumulator data, ALU result, or zero. The PC and operand are only 4 bits, so they are zero-extended to 8 bits before being placed on the bus.

```
case (sel)
    BUS_PC:      bus_data = {4'h0, pc};
    BUS_RAM:     bus_data = ram_data;
    BUS_OPERAND: bus_data = {4'h0, ir_operand};
    BUS_A:       bus_data = a_data;
    BUS_ALU:     bus_data = alu_result;
    default:     bus_data = 8'h00;
endcase
```

Listing 11: Bus source selection from bus_mux.v

3 Instruction Set and Opcode Format

Each instruction is 8 bits:

$$\text{Instruction} = \{\text{opcode}[3:0], \text{operand}[3:0]\}$$

Opcode	Instruction	Operation
1	LDA addr	A = RAM[addr]
2	ADD addr	A = A + RAM[addr]
3	SUB addr	A = A - RAM[addr]
4	MUL addr	A = A * RAM[addr] using shift-add logic
5	DIV addr	A = A / RAM[addr] using shift/subtract division
E	OUT	Output = A
F	HLT	Stop CPU execution
0	NOP	No operation/reserved

For example, `8'h4B` is interpreted as `MUL RAM[0xB]`.

4 ALU Design

The ALU in `alu.v` is a combinational module. It receives the accumulator value `a`, the B register value `b`, and the current opcode. From these inputs, it continuously produces an 8-bit `result` and a `div_zero` flag. The result is only stored back into the accumulator when the control unit reaches the `ALU_WRITE` state.

```
module alu(
    input [7:0] a,
    input [7:0] b,
    input [3:0] opcode,
    output reg [7:0] result,
    output reg div_zero
);
```

Listing 12: ALU module interface

4.1 ADD and SUB

ADD and SUB use standard 8-bit binary arithmetic. ADD computes `A + B`, while SUB computes `A - B`. Because the datapath is 8 bits wide, overflow and underflow wrap around naturally. For example, `250 + 10` becomes `4`, and `3 - 10` becomes `249`.

```
case (opcode)
    OP_ADD:
        result = a + b;
    OP_SUB:
        result = a - b;
endcase
```

Listing 13: ADD and SUB selection

4.2 Multiplication

MUL uses the shift-and-add multiplication algorithm. The multiplier is checked bit by bit. If a multiplier bit is 1, the current shifted multiplicand is added to the product. The multiplicand is shifted left every iteration. This avoids the Verilog `*` operator while still describing real multiplication hardware. The internal product is 16 bits, but only the lower 8 bits are returned to match the CPU datapath.

The algorithm begins by clearing the product register. It then scans the multiplier from bit 0 to bit 7. For each bit, the shifted multiplicand is added only when that multiplier bit is high. After each iteration, the multiplicand is shifted left by one position. This is equivalent to binary long multiplication and is suitable for this project because the 8-bit operand width gives a fixed loop count of eight iterations.

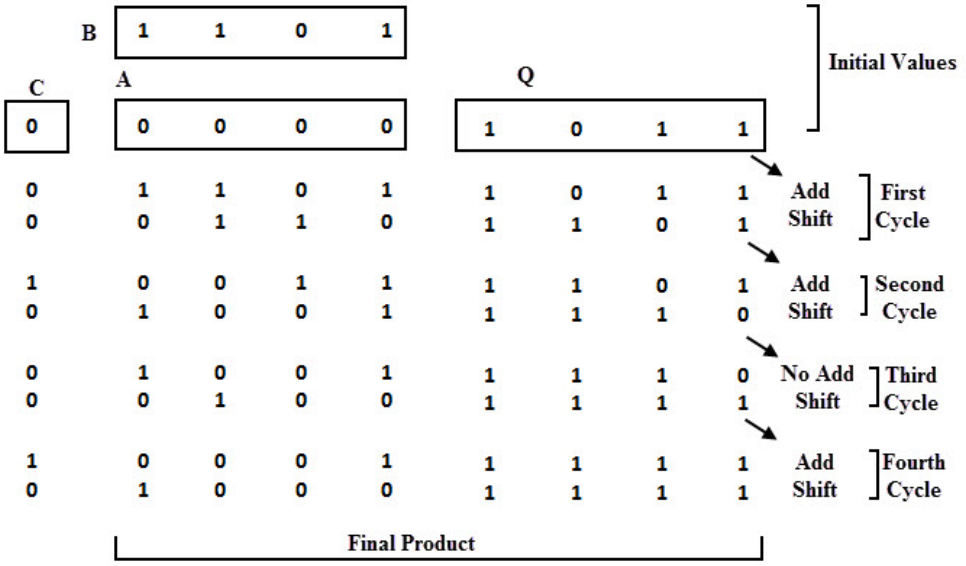

Figure 1: Shift-and-add multiplication method illustrated with a 4-bit example

Although Figure 1 illustrates the method using a smaller 4-bit example, the implemented Verilog function applies the same shift-and-add process to 8-bit operands using eight fixed iterations.

```
product = 16'h0000;
shifted_multiplicand = {8'h00, multiplicand};

for (i = 0; i < 8; i = i + 1) begin
    if (multiplier[i])
        product = product + shifted_multiplicand;
    shifted_multiplicand = shifted_multiplicand << 1;
end

multiply_shift_add = product[7:0];
```

Listing 14: Shift-and-add multiplication from alu.v

4.3 Division

DIV uses the restoring division algorithm. The quotient is built one bit at a time from the most significant bit to the least significant bit. At each step, the partial remainder is shifted left and compared with the divisor. If the partial remainder is large enough, the divisor is subtracted and the quotient bit is set to 1. This avoids the Verilog `/` operator and produces integer division, so any remainder is discarded.

The algorithm first initializes the quotient and remainder to zero. It then processes the dividend from bit 7 down to bit 0. At every iteration, the next dividend bit is shifted into the partial remainder. If the partial remainder is greater than or equal to the divisor, the divisor is subtracted and the corresponding quotient bit is set. Otherwise, the quotient bit remains zero. This produces an integer quotient and discards the final remainder.

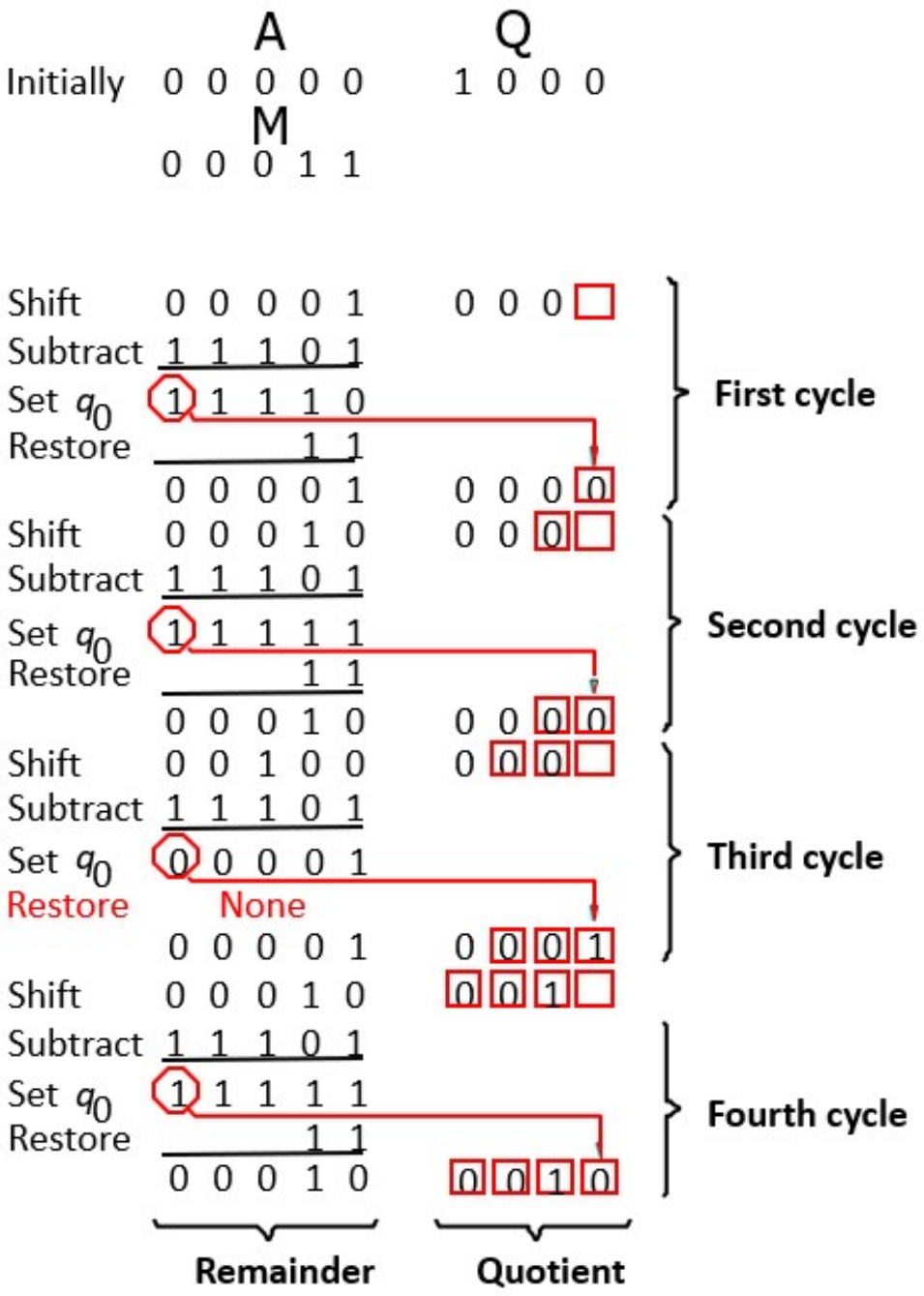

Figure 2: Restoring division algorithm used for DIV

```
for (i = 7; i >= 0; i = i - 1) begin
    remainder = {remainder[7:0], dividend[i]};
    if (remainder >= {1'b0, divisor}) begin
        remainder = remainder - {1'b0, divisor};
        quotient[i] = 1'b1;
    end
end
```

Listing 15: Restoring division from alu.v

4.4 Division by Zero

If the divisor is zero, the ALU returns `8'hFF` and asserts `div_zero`:

```
if (b == 8'h00) begin
    result = 8'hFF;
    div_zero = 1'b1;
end else begin
    result = divide_restoring(a, b);
end
```

Listing 16: Division-by-zero handling

4.5 CPU-Level `div_zero` Latch

The `div_zero` signal is handled in two levels. First, the ALU generates a raw combinational `div_zero` signal when the current operation is DIV and the divisor is zero. Then, `sap1_top.v` latches this signal as the CPU-level `div_zero` flag only when the ALU result is written back to the accumulator.

```
always @(posedge clk or posedge reset) begin
    if (reset)
        div_zero <= 1'b0;
    else if (a_load && bus_sel == BUS_ALU)
        div_zero <= (alu_op == OP_DIV) ? alu_div_zero : 1'b0;
end
```

Listing 17: CPU-level div_zero latch in sap1_top.v

This design is used because the ALU is combinational, so its raw flag can change whenever the ALU inputs change. The CPU should record a divide-by-zero event only when the DIV result is actually committed during `ALU_WRITE`. Keeping this latch in the top-level datapath also keeps the control unit focused on FSM sequencing instead of storing datapath error flags.

4.6 Opcode-Based Result Selection

The final ALU output is selected using the opcode. For unsupported opcodes, the ALU passes the accumulator value through unchanged.

```
case (opcode)
    OP_ADD: result = a + b;
    OP_SUB: result = a - b;
    OP_MUL: result = multiply_shift_add(a, b);
    OP_DIV: begin
        if (b == 8'h00) begin
            result = 8'hFF;
            div_zero = 1'b1;
        end else begin
            result = divide_restoring(a, b);
        end
    end
    default: result = a;
endcase
```

Listing 18: Complete ALU operation selection

5 Control Unit FSM

The control unit is a finite state machine. It is responsible for giving the datapath its timing: selecting the correct bus source, enabling the correct register, incrementing the PC, and stopping the CPU when HLT is reached.

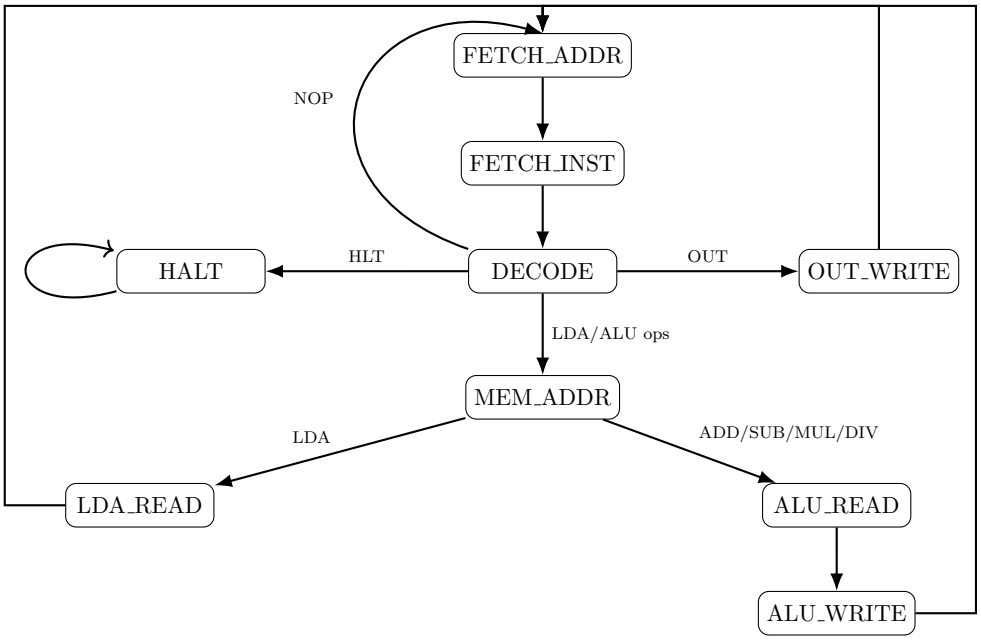

Figure 3: FSM state diagram of the SAP-1 control unit

State	Purpose
FETCH_ADDR	Put PC on bus and load MAR.
FETCH_INST	Read RAM[MAR] into IR and increment PC.
DECODE	Select the execution path from opcode.
MEM_ADDR	Load operand address into MAR.
LDA_READ	Load RAM data into Accumulator.
ALU_READ	Load RAM data into B Register.
ALU_WRITE	Write ALU result back to Accumulator.
OUT_WRITE	Copy Accumulator to Output Register.
HALT	Assert halted and stop sequencing.

```
FETCH_ADDR: begin
    bus_sel = BUS_PC;
    mar_load = 1'b1;
    next_state = FETCH_INST;
end

FETCH_INST: begin
    bus_sel = BUS_RAM;
    ir_load = 1'b1;
    pc_inc = 1'b1;
    next_state = DECODE;
end

DECODE: begin
    case (opcode)
        OP_HLT: next_state = HALT;
        OP_OUT: next_state = OUT_WRITE;
        OP_NOP: next_state = FETCH_ADDR;
        default: next_state = MEM_ADDR;
    endcase
end
```

Listing 19: Fetch and decode examples from control_unit.v

6 Simulation Results

6.1 Main Program Test

The testbench loads this program into RAM:

```
write_memory(4'h0, 8'h18); // LDA RAM[0x8]
write_memory(4'h1, 8'h29); // ADD RAM[0x9]
write_memory(4'h2, 8'h3A); // SUB RAM[0xA]
write_memory(4'h3, 8'h4B); // MUL RAM[0xB]
write_memory(4'h4, 8'h5C); // DIV RAM[0xC]
write_memory(4'h5, 8'hE0); // OUT
write_memory(4'h6, 8'hF0); // HLT
```

Listing 20: Main program loaded in sap1_tb.v

The data values are RAM[8] = 10, RAM[9] = 3, RAM[A] = 4, RAM[B] = 2, and RAM[C] = 3. Therefore:

$$((10 + 3 - 4) \times 2)/3 = 6$$

6.2 Waveform Results

The main waveform was captured from approximately 56 ns to 130 ns. This interval shows the complete execution sequence of the main program: LDA, ADD, SUB, MUL, DIV, OUT, and HLT. The waveform includes the following key signals:

`clk`	`reset`	`out`
`halted`	`div_zero`	`cpu_state`
`cpu_pc`	`cpu_mar`	`cpu_instruction`
`cpu_a`	`cpu_b`	`cpu_alu_result`
`cpu_bus`		

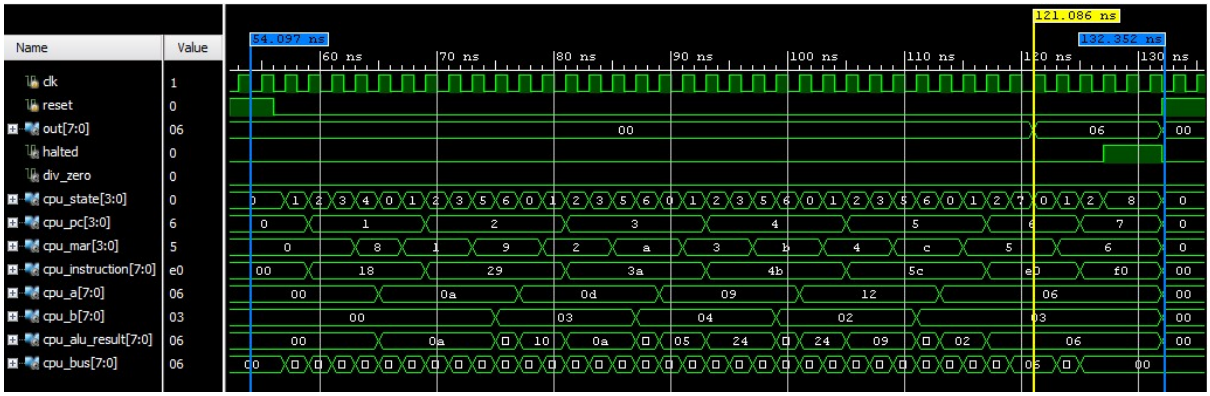

Figure 4: Main program waveform showing LDA, ADD, SUB, MUL, DIV, OUT, and HLT

Figure 4 shows the complete execution of the main program. After `reset` goes low, the CPU begins at address `0`. The `cpu_pc` signal then increases step by step as the CPU fetches each instruction. The instruction values appear in this order: `18`, `29`, `3A`, `4B`, `5C`, `E0`, and `F0`. These values match the program sequence `LDA 8`, `ADD 9`, `SUB A`, `MUL B`, `DIV C`, `OUT`, and `HLT`.

The accumulator value `cpu_a` shows the result after each operation. First, `LDA 8` loads RAM[8] = `0A`, so the accumulator becomes 10. Then `ADD 9` adds RAM[9] = `03`,

producing `0D`, or 13. Next, `SUB A` subtracts RAM[A] = `04`, so the accumulator becomes `09`. After that, `MUL B` multiplies by RAM[B] = `02`, giving `12`, or 18. Finally, `DIV C` divides by RAM[C] = `03`, so the accumulator becomes `06`.

The `cpu_bus` signal shows the value currently being transferred inside the datapath. Its value changes often because different FSM states select different bus sources, such as the PC, RAM output, instruction operand, accumulator, or ALU result. The `cpu_alu_result` signal also changes whenever the ALU inputs change because the ALU is combinational. However, the ALU result is stored into the accumulator only during the `ALU_WRITE` state.

At the `OUT` instruction, the final accumulator value `06` is copied to the output register, so `out` becomes `06`. When `HLT` is reached, the `halted` signal becomes high and the CPU stops executing. The `div_zero` signal stays low throughout this waveform because the division operand is not zero.

6.3 Divide-by-Zero Waveform

A second waveform was captured from approximately 162 ns to 199 ns. This interval verifies the defined divide-by-zero behavior of the CPU-level DIV instruction.

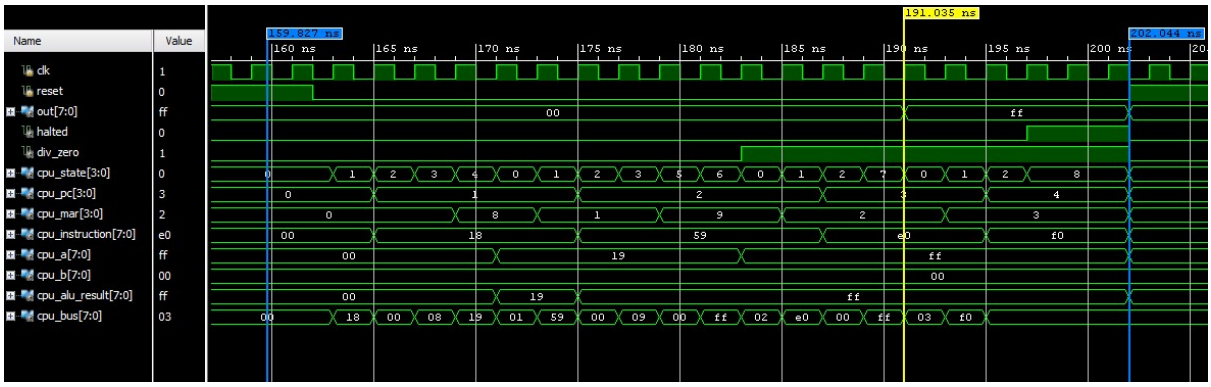

Figure 5: Divide-by-zero waveform showing result `8'hFF` and asserted `div_zero`

For the divide-by-zero case, the program loads 25 into the accumulator and then executes `DIV RAM[0x9]`, where RAM[9] contains zero. The ALU returns `8'hFF`, and the `div_zero` flag is asserted. After the OUT instruction, the output register also shows `FF`, confirming that the defined error behavior is visible at the CPU output.

6.4 Test Summary

Simulation confirms that the output register becomes `8'd6`, the CPU halts, and `div_zero` remains low for the normal program. Besides the main waveform test, the testbench also includes additional cases to check important edge behavior:

Test Case	Expected Behavior
Division by zero	ALU returns `8'hFF` and asserts `div_zero`.
Multiplication overflow	Result wraps around to the lower 8 bits.
Addition overflow	Result wraps around within the 8-bit datapath.
Subtraction underflow	Result wraps around using 8-bit arithmetic.
Integer division truncation	Remainder is discarded.
LDA overwrite	New loaded value replaces the old accumulator value.
HLT hold	CPU remains halted after the HLT instruction.

All test cases pass in XSim.

7 Design Decisions, Challenges, and Conclusion

One important design decision was how to organize the shared bus. A tri-state bus is close to the traditional SAP-1 explanation and is easy to understand conceptually because each component can appear to "drive" the same bus. However, internal tri-state buses are not ideal for FPGA synthesis and can create multiple-driver problems if the control signals are not handled carefully. Therefore, this project uses a bus multiplexer instead. The trade-off is that the mux version is slightly less similar to the textbook diagram, but it is clearer in Verilog, safer for synthesis, and easier to debug in simulation. Another related decision was to latch the CPU-level `div_zero` flag in `sap1_top.v` rather than in the control unit. Since `div_zero` describes the result of a datapath operation, it is stored when the ALU result is committed to the accumulator, while the control unit remains responsible only for FSM sequencing.

The main challenges were learning to write Verilog correctly and translating algorithms into circuit-style logic. MUL and DIV could not use `*` and `/`, so the arithmetic algorithms had to be rewritten as fixed hardware procedures. It also took time to understand how each module, such as the PC, MAR, IR, registers, ALU, RAM, bus, and control unit, works together across multiple clock cycles. These difficulties were solved by applying the processor datapath and control concepts studied in Module 2 and by testing each part through simulation.

In conclusion, the project successfully implements a modified SAP-1 CPU with a clear multi-cycle datapath and FSM-based control unit. The CPU supports memory access, arithmetic operations, output, and halt behavior using separate Verilog modules. Simulation results confirm that the design works correctly, including the added MUL and DIV instructions and important edge cases such as division by zero. Overall, the project helped connect the theory of CPU organization with an actual hardware description implementation.